

LLM 활용 소프트웨어

☰ 담당자	
⚙ 상태	시작 전
📅 제출일	@1월 12
Σ d-day	마감

LLM 활용 소프트웨어

1. 개요

본 소프트웨어는 **LLM**을 활용하여 사용자의 자연어 질문을 분석하고, 문서 데이터베이스에 저장된 정보를 기반으로 답변을 생성하는 **문서 기반 질의응답 소프트웨어**입니다.

LLM은 벡터 데이터베이스(Vector DB)와 결합된 **RAG(Retrieval-Augmented Generation)** 구조로 활용되며, 임의의 생성이 아닌 **문서 근거 중심의 응답 생성**을 목표로 합니다.

2. LLM 활용 구조 (현재)

현재 구현된 시스템에서 LLM은 다음과 같은 방식으로 활용됩니다.

- 벡터 데이터베이스에 저장된 문서 임베딩과의 유사도 검색 수행
- 검색된 문서를 입력 컨텍스트로 하여 답변 문장 생성

LLM은 **판단 로직을 담당하지 않으며**, 검색 결과를 기반으로 자연어 응답을 생성하는 역할만 수행합니다.

LLM은 openAI API `gpt-4o-mini`를 사용합니다.

`generator.py`

```
"""
목소리 프로젝트 - RAG Answer Generator
LLM 기반 구조화된 답변 생성
"""

import os
import time
from typing import List, Dict, Tuple, Optional
from openai import OpenAI

DISCLAIMER = "본 답변은 정보 제공 목적이며 법률 자문이 아닙니다. 최종 판단·결정은 관련 기관 또는 전문가와 상담하여 진행해 주세요."
```

```

SECTIONS = [
    "1. 추천 기관 및 사유",
    "2. 유사 사례",
    "3. 관련 법적 근거",
    "4. 다음 행동 체크리스트"
]

```

```

class RAGGenerator:

```

```

    """

```

```

    LLM 기반 답변 생성기

```

```

    Generates structured answers with:

```

- Fixed disclaimer
- Agency recommendation + reason
- Similar cases (2-3 with sources)
- Legal basis (criteria/laws with citations)
- Next action checklist

```

    """

```

```

def __init__(self, model: str = "gpt-4o-mini", use_llm: bool = True):

```

```

    """

```

```

    Args:

```

```

        model: LLM 모델 (기본값: gpt-4o-mini)
        use_llm: LLM 사용 여부 (False면 stub 모드)

```

```

    """

```

```

        self.model = model

```

```

        self.use_llm = use_llm and bool(os.getenv('OPENAI_API_KEY'))

```

```

        if self.use_llm:

```

```

            self.client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

```

```

def generate_answer(self, query: str, chunks: List[Dict]) → Dict:

```

```

    """

```

```

    검색 결과 기반 구조화된 답변 생성

```

```

    Args:

```

```

        query: 사용자 질문
        chunks: 검색된 청크 리스트 (SearchResult dict 형식)

```

```

    Returns:

```

```

        {
            'answer': str,
            'chunks_used': int,
            'model': str,

```

```

        'has_sufficient_evidence': bool,
        'clarifying_questions': List[str]
    }
    """
    if not chunks:
        return self._no_results_response()

    if self.use_llm:
        return self._generate_llm_answer(query, chunks)
    else:
        return self._generate_stub_answer(query, chunks)

def generate_answer_instrumented(self, query: str, chunks: List[Dict]) → Dict:
    """
    검색 결과 기반 구조화된 답변 생성 with logging metadata

    Returns standard fields plus:
    - 'system_prompt': str
    - 'user_prompt': str
    - 'prompt_tokens': int
    - 'completion_tokens': int
    - 'response_time_ms': float
    """
    if not chunks:
        result = self._no_results_response()
        result.update({
            'system_prompt': '',
            'user_prompt': '',
            'prompt_tokens': 0,
            'completion_tokens': 0,
            'response_time_ms': 0
        })
        return result

    if not self.use_llm:
        result = self._generate_stub_answer(query, chunks)
        result.update({
            'system_prompt': '',
            'user_prompt': '',
            'prompt_tokens': 0,
            'completion_tokens': 0,
            'response_time_ms': 0
        })
        return result

```

```

# LLM mode with instrumentation
system_prompt = self._get_system_prompt()
user_prompt = self._build_prompt(query, chunks)

start = time.time()
response = self.client.chat.completions.create(
    model=self.model,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    temperature=0.3
)
response_time = (time.time() - start) * 1000

# Extract token usage
usage = response.usage
prompt_tokens = usage.prompt_tokens if usage else 0
completion_tokens = usage.completion_tokens if usage else 0

answer_text = response.choices[0].message.content

# Safety guardrails
has_evidence, questions = self._check_evidence(query, chunks)

if not has_evidence:
    answer_text = self._format_insufficient_evidence(answer_text, questions)

return {
    'answer': answer_text,
    'chunks_used': len(chunks),
    'model': self.model,
    'has_sufficient_evidence': has_evidence,
    'clarifying_questions': questions,
    # Logging metadata
    'system_prompt': system_prompt,
    'user_prompt': user_prompt,
    'prompt_tokens': prompt_tokens,
    'completion_tokens': completion_tokens,
    'response_time_ms': response_time
}

def _generate_llm_answer(self, query: str, chunks: List[Dict]) → Dict:
    """Generate answer using LLM with S1-1 template"""
    prompt = self._build_prompt(query, chunks)

```

```

response = self.client.chat.completions.create(
    model=self.model,
    messages=[
        {"role": "system", "content": self._get_system_prompt()},
        {"role": "user", "content": prompt}
    ],
    temperature=0.3
)

answer_text = response.choices[0].message.content

# Safety guardrails
has_evidence, questions = self._check_evidence(query, chunks)

if not has_evidence:
    answer_text = self._format_insufficient_evidence(answer_text, questions)

return {
    'answer': answer_text,
    'chunks_used': len(chunks),
    'model': self.model,
    'has_sufficient_evidence': has_evidence,
    'clarifying_questions': questions
}

```

```

def _get_system_prompt(self) → str:
    return """당신은 한국 소비자 분쟁 조정 전문 상담 어시스턴트입니다.

```

역할:

- 검색된 사례와 법령을 기반으로 정보를 제공합니다
- 법률 자문이나 확정적인 판단을 하지 않습니다
- 근거가 부족할 경우 추가 질문을 통해 정보를 수집합니다

답변 형식:

```

""" + DISCLAIMER + "\n\n" + "\n".join(SECTIONS) + """

```

금지 사항:

- "~해야 합니다", "~입니다" 같은 단정적 표현
 - 법률 판단이나 예측
 - 개인정보 요구
- ```

"""

```

```

def _generate_stub_answer(self, query: str, chunks: List[Dict]) → Dict:
 """Stub mode (no LLM) - structured format with citations"""

```

```

lines = [DISCLAIMER, "\n"]

lines.append("1. 추천 기관 및 사유")
lines.append("현재 LLM 연동 전 단계입니다. 검색 결과를 참고하세요.\n")

lines.append("2. 유사 사례")
for i, chunk in enumerate(chunks[:3], 1):
 lines.append(f"{i}. [{chunk.get('source_org', '알 수 없음')}] {chunk.get('doc_title', '제목 없음')}")
 lines.append(f" 유사도: {chunk.get('similarity', 0):.2f}")
 if chunk.get('decision_date'):
 lines.append(f" 결정일: {chunk['decision_date']}")
 if chunk.get('url'):
 lines.append(f" 출처: {chunk['url']}")
 lines.append("")

lines.append("3. 관련 법적 근거")
lines.append("LLM 연동 후 관련 법령 및 기준이 표시됩니다.\n")

lines.append("4. 다음 행동 체크리스트")
lines.append("□ 관련 서류 준비 (영수증, 계약서 등)")
lines.append("□ 해당 기관 연락처 확인")
lines.append("□ 사실관계 정리")

return {
 'answer': "\n".join(lines),
 'chunks_used': len(chunks),
 'model': 'stub',
 'has_sufficient_evidence': True,
 'clarifying_questions': []
}

```

### 3. 멀티 에이전트 기반 확장 설계 (예정)

본 소프트웨어는 향후 **멀티 에이전트** 구조로 확장될 예정이며, 각 단계별 역할을 분리하여 오케스트레이션 방식으로 관리합니다.

#### 예정 에이전트 구성

##### 1. 질의 분석 에이전트

- 사용자 입력을 분석하여 질의 의도 및 핵심 정보 추출, 쿼리 재생성

##### 2. 정보 검색 에이전트

- 벡터 DB 및 관계형 DB를 조회하는 유일한 에이전트
- 문서 검색 및 관련 정보 수집 담당

### 3. 답변 생성 에이전트

- 검색된 문서를 기반으로 LLM을 활용한 답변 생성

### 4. 답변 검토 에이전트

- 생성된 답변의 문서 근거 여부 및 응답 품질 검토

각 에이전트는 역할이 분리되어 있으며, 중앙 오케스트레이션 로직에 의해 호출됩니다.

---

## 4. 소스 코드 정보

- **GitHub** 저장소 주소

```
https://github.com/SKN19-FINAL-5team/LLM
```

소스 코드에는 문서 임베딩 처리, LLM 호출 로직, 검색 결과를 활용한 답변 생성 흐름이 포함되어 있습니다.